

Rethinking LoRA Placement at the Vocabulary Boundary with A-LoRA

Anonymous ACL submission

Abstract

Low-Rank Adaptation (LoRA) is typically applied to internal Transformer layers, while vocabulary-boundary layers such as input embeddings and output LM heads have long lacked systematic evaluation. We revisit this placement question and propose **A-LoRA**, a hidden-space parameterization that does not learn free residuals directly on $V \times d$ vocabulary matrices, but instead induces vocabulary-matrix updates via a shared low-rank affine transform, so parameter count depends mainly on hidden dimension d rather than vocabulary size V . Analyzing 30 Base→Instruct model pairs, we find that output LM heads and tied shared matrices better match this affine structure, whereas untied input embeddings should not be treated symmetrically under the same assumption. Downstream LoRA post-training further shows that optimal vocabulary-boundary placement is conditional: A-LoRA as a standalone small-budget adapter is stronger on the input side; when combined with standard hidden LoRA, the `lm_head` side becomes a more stable low-cost supplement. Our results indicate that vocabulary-boundary layers should not be excluded from LoRA design space by default, but their topology should adapt to existing adaptation capacity rather than being added symmetrically to input and output sides unconditionally.

1 Introduction

As large language models grow in scale, full fine-tuning incurs substantial cost in storage, compute, and multi-task model maintenance. Low-Rank Adaptation (LoRA) has become one of the most widely used parameter-efficient fine-tuning methods for large models by freezing pretrained weights and introducing trainable low-rank residuals (Hu et al., 2022). Original LoRA inserts adapters mainly in attention and MLP projections; common PEFT implementations and default configurations likewise focus on internal Transformer blocks

(Mangrulkar et al., 2022; Hugging Face, 2026a). LoRA placement has therefore long centered on internal layers rather than input embeddings and output LM heads. We refer to both as *vocabulary-boundary layers*, because they connect discrete token space to continuous hidden representation space at the model’s entry and exit.

In settings such as domain terminology, new tokens, multilingual transfer, vocabulary expansion, or continued pretraining, input embeddings and output LM heads can still affect adaptation, and both admit standard low-rank residuals. Yet they are rarely evaluated systematically in default LoRA workflows, mainly due to three obstacles: Vocab LoRA on a single matrix costs roughly $r(V + d)$ parameters and grows linearly with V ; weight tying, vocabulary-parallel tensor parallelism, and weight merging make deployment more cumbersome than for internal layers; and the assumption that vocabulary-boundary layers are “stable after pretraining” is largely an empirical belief not yet validated under a unified budget. We therefore ask: are vocabulary-boundary adapters worth training, and must they take the form of Vocab LoRA that grows with V ?

To reduce parameter and storage cost at the vocabulary boundary, we propose **A-LoRA**, a hidden-space parameterization. In its minimal form, for a frozen vocabulary matrix $M_0 \in \mathbb{R}^{V \times d}$, rather than learning a free $V \times d$ residual, we learn hidden-dimensional low-rank factors P, Q and shift β , yielding $M_{\text{eff}} \approx M_0(I + \frac{\alpha}{r}PQ) + \mathbf{1}_V\beta^\top$. A-LoRA’s trainable parameter count therefore depends mainly on hidden dimension d rather than vocabulary size V ; full forward paths, parameter counts, and topologies are formalized in §4. We also analyze 30 Base→Instruct model pairs and find a marked asymmetry in post-training change structure between output LM heads / tied shared matrices and untied input embeddings (§3).

Under a unified budget, we further com-

pare Vocab LoRA and A-LoRA across four topologies—input-only, output-only, shared, and decoupled—against internal-layer LoRA and parameter-matched baselines (§2, §4, and the experiments section; we do not benchmark inference latency or serving throughput). Our main contributions are: (1) framing input embeddings and output LM heads as a unified vocabulary-boundary placement problem and systematically comparing adaptation choices across input-side, output-side, and tied/untied topologies; (2) proposing A-LoRA, a hidden-space affine parameterization that reduces vocabulary-boundary LoRA parameter cost, and establishing its structural applicability conditions from 30 Base→Instruct model pairs; and (3) demonstrating through downstream LoRA post-training that placement is jointly determined by target residual type and existing adaptation capacity: A-LoRA is stronger on the input side when used standalone, while the lm_head side is better suited as a low-cost supplement when combined with hidden LoRA; for tied models, shared mergeable A-LoRA preserves tied structure while maintaining performance.

2 Related Work

2.1 LoRA and Adapter Placement

LoRA achieves parameter-efficient downstream adaptation by freezing pretrained weights and training low-rank residuals (Hu et al., 2022). Subsequent methods such as QLoRA and AdaLoRA further improve quantization, training efficiency, and rank allocation (Dettmers et al., 2023; Zhang et al., 2023), and mainstream PEFT libraries have consolidated mature implementations around these adapters (Mangrulkar et al., 2022; Hugging Face, 2026a). Existing research and engineering defaults focus mainly on attention and MLP projections inside Transformer blocks, and rarely treat input embeddings and output LM heads as core variables of LoRA *placement*. Multiplicative LoRA variants such as LoRMA₊ also exist (Bihany et al., 2025), but their target remains internal-layer parameterization; we discuss how A-LoRA relates to them in Appendix A.8.

2.2 Vocabulary-Boundary Adaptation

Input embeddings and output LM heads connect discrete token space and continuous hidden representation space, and can therefore affect adaptation in settings such as new special tokens, vo-

cabulary expansion, domain terminology, or multilingual transfer. Existing vocabulary-related practice is mostly **scenario-driven**: for example, training partial embeddings when adding special tokens or using PEFT Trainable Tokens (Hugging Face, 2026b), or saving/updating vocabulary-boundary weights during vocabulary expansion (Hugging Face, 2026a). These practices show that vocabulary parameters can be trained, but their goal is usually local vocabulary editing or engineering need, rather than systematically comparing full input embedding / output LM head as LoRA placement against internal-layer baselines under a unified budget.

2.3 Weight Tying and Deployment Constraints

Input/output weight tying is a common parameter-sharing strategy in language models (Press and Wolf, 2017; Inan et al., 2017). In tied decoder-only LLMs, embedding and LM head share an underlying vocabulary matrix ($E = U$), so vocabulary-boundary adapters naturally raise topology choices such as shared, unilateral, and bilateral decoupled designs. Large-vocabulary models also often involve vocabulary-parallel tensor parallelism and weight-merge constraints, making Vocab LoRA on $V \times d$ matrices less plug-and-play than internal adapters. We note these factors as design constraints but do not expand them into a systems engineering benchmark.

In summary, the LoRA ecosystem has studied internal adapters extensively, but whether vocabulary-boundary layers are worth adapting, which side to adapt, and which parameterization to use under large-vocabulary cost and tied/untied constraints still lack unified answers.

3 Post-Training Vector-Space Analysis and When A-LoRA Applies

The appeal of A-LoRA is that parameter count does not grow with vocabulary size V . But fewer parameters alone do not justify the parameterization: because A-LoRA constrains vocabulary-matrix updates to a shared affine transform induced in hidden space, it should be used only when the target matrix’s post-training change is itself affine-friendly. This section therefore asks whether vocabulary row vectors during Base→Instruct transition conform to this structure. The core finding is asymmetric: output LM heads and tied shared matrices satisfy this structure, while untied input embeddings largely

do not.

This result provides a structural criterion for using A-LoRA: it is better suited for vocabulary matrices whose post-training change matches hidden-space affine form, and should not be applied symmetrically to all vocabulary-boundary layers merely because it uses fewer parameters. Full per-model numbers, auxiliary metrics, and budget-probe protocols appear in Sections A.2, A.4 and A.5.

3.1 When Is A-LoRA a Reasonable Parameterization?

For a vocabulary-boundary matrix $X \in \mathbb{R}^{V \times d}$ (V vocabulary size, d hidden dimension), let Base weights be X , Instruct weights be Y , and the change matrix be

$$\Delta W = Y - X. \quad (1)$$

Here ΔW describes drift of the same vocabulary row vectors in hidden space before and after post-training. In untied models, input embedding matrix X_E and output LM head matrix X_U are separate; in tied models $X_E = X_U$ is the shared matrix, and we report only the E=U shared row below.

Vocab LoRA learns a low-rank residual directly on the $V \times d$ vocabulary matrix; A-LoRA assumes vocabulary row changes can be induced jointly by the row structure of X and a shared hidden-dimensional transform. In other words, we ask whether an important part of ΔW can be written as

$$\Delta W \approx X(A - I) + \mathbf{1}_V b^\top, \quad (2)$$

where $A \in \mathbb{R}^{d \times d}$, $b \in \mathbb{R}^d$, and $\mathbf{1}_V \in \mathbb{R}^V$ is the all-ones column vector. This condition does not require all post-training change to be explained affinely; it requires only that important change components align with the affine structure induced by X . The substantive question in this section is therefore which of embedding, LM head, and tied shared matrix actually satisfy this structural condition.

3.2 Measuring Affine-Friendly Structure

Examining the full affine subspace directly is too broad. We therefore keep only high-energy directions induced by X and the bias term. Let design matrix $Z = [X, \mathbf{1}_V] \in \mathbb{R}^{V \times (d+1)}$, perform SVD on Z , and take the first K left singular directions Q_K whose cumulative energy reaches $\eta = 95\%$; $P_K = Q_K Q_K^\top$ projects onto this effective affine

Table 1: Median effective affine subspace explained variance (30 BI pairs, $\eta = 95\%$). $R_K^2 \equiv R_{\text{aff},K}^2$; Untied reports E/U separately; Tied reports shared matrix only.

Group	n	R_K^2
Untied E	13	3.6%
Untied U	13	33.0%
Tied E=U	17	32.2%

subspace. The main metric is

$$R_{\text{aff},K}^2 := \frac{\|P_K \Delta W\|_F^2}{\|\Delta W\|_F^2}, \quad (3)$$

which measures what fraction of post-training change energy lies in the effective affine structure of X . The main text uses this metric as the primary criterion; auxiliary metrics such as principal-direction alignment, random baselines, and centered decomposition appear in Sections A.4 and A.13, with conclusions consistent with the grouped results below.

3.3 Affine-Friendly Structure Appears on LM Head / Tied Matrix, Not Input Embedding

The key finding is a clear structural split: output LM heads and tied shared matrices are affine-friendly; untied input embeddings are not. We compute the above metrics on non-excluded 30 Base→Instruct (BI) pairs (17 tied + 13 untied); untied models report E and U separately, tied models report the shared matrix. Table 1 gives median results.

On Untied U, $R_{\text{aff},K}^2 \approx 33\%$: roughly one third of total change energy lies in the effective affine subspace. By contrast, on Untied E, only about 4%. This shows U-side post-training change is more induced by base vocabulary geometry, while E-side change is more diffuse and does not support symmetric application of the same parameterization to input embedding. Auxiliary principal-direction alignment metrics in the appendix show the same split: U/tied are well above random baseline, while E is near random.

Shared matrices in tied models behave like untied U rather than untied E: $R_{\text{aff},K}^2 \approx 32\%$, on par with the U side. Affine-friendliness is therefore not unique to untied U, but associated with the LM head side / shared weight matrix; after untied splitting the property remains mainly on U, while tied models express it uniformly on the shared matrix.

This split also cannot be explained by “which side changes more.” In all 13/13 untied pairs,

$R_{\text{aff},K}^2(\text{U}) > R_{\text{aff},K}^2(\text{E})$; on DeepSeek-V3, U-side $\|\Delta W\|$ is only $0.4\times$ that of E, yet U/E $R_{\text{aff},K}^2$ is still about $2.7\times$. Metric (3) normalizes by $\|\Delta W\|$, so the E/U split reflects structural differences between change direction and base vocabulary geometry.

3.4 From Structural Match to Budget Advantage and Placement Strategy

If a target matrix satisfies the affine-friendly condition, the same trainable parameter budget is better spent on hidden-space affine rank than on vocabulary-dimensional W-form rank. The reason is direct: Vocab LoRA learns residuals on $V \times d$ matrices and parameter count grows with V ; A-LoRA learns low-rank affine factors in hidden dimension and grows mainly with d . Matching parameter counts approximately yields

$$r_{\text{aff}} \approx r_W \cdot \frac{V+d}{2d} \approx r_W \cdot \frac{V}{2d}. \quad (4)$$

For typical LLMs, $V/d \sim 15\text{--}120$, so **Vocab LoRA rank=1 is roughly equivalent in budget to A-LoRA hidden rank $\sim 8\text{--}61$** (median about 15 on untied U). On affine-friendly U/tied sides, A-LoRA can therefore cover more hidden-space affine directions at equal budget; on non-affine-friendly E sides, this budget advantage should not automatically translate into placement advice.

Offline fitting probes confirm this. Given observed post-training change ΔW , we approximate it under equal budget with Vocab LoRA and A-LoRA functional forms, using explained gain to simulate expressivity for the target change; this probe does not train adapters and does not replace downstream experiments. Results show median aff/W @ $k=1$ of **2.18 $\times$** on Untied U (12/13 wins), **3.00 $\times$** on Tied (E=U) (15/17 wins), but only $0.34\times$ on Untied E (1/13 wins). Budget advantage is therefore not universal compression gain, but match gain triggered by affine-friendly structure.

The resulting placement implications are conditional: in untied models, prioritize A-LoRA on the LM head side and do not apply it symmetrically on input embedding just because matrix shapes match; in tied models, the shared matrix shows U-like affine properties, so shared A-LoRA matches update structure better than shared Vocab LoRA and more naturally preserves merge back to a single matrix; if using input-output decoupled topology, it should reflect the mechanistic split of

affine-friendly U and diffuse E, rather than treating both sides as equivalent placements.

4 Method

This section formalizes two vocabulary-boundary parameterizations (Vocab LoRA and A-LoRA) and four placement topologies (input-only, output-only, shared, and decoupled). The former determines how vocabulary-matrix updates are expressed; the latter determines where updates are placed. Implementation details appear in Sections A.8 and A.12.

4.1 Problem Setup

Consider a decoder-only LLM with vocabulary size V and hidden dimension d . Let input embedding matrix be $E_0 \in \mathbb{R}^{V \times d}$ and output LM head matrix be $U_0 \in \mathbb{R}^{V \times d}$; when tied, $E_0 = U_0 = W_0$. In the PEFT setting, base weights are frozen and only adapter parameters Θ are trained.

For any vocabulary matrix $M_0 \in \{E_0, U_0, W_0\}$, the adapter yields effective matrix

$$M_{\text{eff}} = M_0 + \Delta M(\Theta). \quad (5)$$

The input side uses E_{eff} for lookup; the output side uses U_{eff} to compute logits. This paper defines only two forms of ΔM at the vocabulary boundary; internal-layer baselines use standard attention/MLP LoRA (Hu et al., 2022).

4.2 Vocab LoRA (W-Form Baseline)

Standard Vocab LoRA learns an additive low-rank residual on the $V \times d$ vocabulary matrix (Hu et al., 2022):

$$M_{\text{eff}} = M_0 + \frac{\alpha}{r} BC, \quad (6)$$

where $B \in \mathbb{R}^{V \times r}$ and $C \in \mathbb{R}^{r \times d}$. Trainable parameter count (including optional bias) is

$$P_W = d + r(V + d). \quad (7)$$

Vocab LoRA does not assume the affine structure of §3 for ΔM , and serves as the direct W-form baseline for testing A-LoRA parameter efficiency; its main cost is that parameters in B grow linearly with V .

4.3 A-LoRA

A-LoRA parameterizes the affine update of §3, $\Delta W \approx X(A - I) + \mathbf{1}_V b^\top$, with low-rank hidden linear factors and a row-shared hidden bias. Let $M_0 \in \mathbb{R}^{V \times d}$,

$$\Delta M_{\text{aff}} = M_0 \cdot \frac{\alpha}{r} PQ + \mathbf{1}_V \beta^\top, \quad (8)$$

Table 2: Four vocabulary-boundary adapter topologies.

ID	Name	E_{eff}	U_{eff}	Role
T1	Input-only	$E_0 + \Delta_E$	U_0	Independent input-side capacity
T2	Output-only	E_0	$U_0 + \Delta_U$	Output-side structural hypothesis
T3	Shared	$W_0 + \Delta$		Tied shared-matrix hypothesis
T4	Decoupled	$E_0 + \Delta_E$	$U_0 + \Delta_U$	Bilateral complement / redundancy

where $P \in \mathbb{R}^{d \times r}$, $Q \in \mathbb{R}^{r \times d}$, and $\beta \in \mathbb{R}^d$. The effective matrix is

$$M_{\text{eff}} = M_0 \left(I_d + \frac{\alpha}{r} PQ \right) + \mathbf{1}_V \beta^\top. \quad (9)$$

In Eq. (8), $M_0(\alpha/r)PQ$ is a low-rank approximation to $X(A - I)$; $\mathbf{1}_V \beta^\top$ is a hidden-dimensional shift shared across rows. Parameter count is

$$P_{\text{aff}} = d + 2dr. \quad (10)$$

Unlike Vocab LoRA, A-LoRA’s multiplicative term couples through M_0 and hidden-side PQ , so update directions are induced by base weights; it should therefore be judged jointly by the structural evidence of §3 and downstream comparisons of §6, rather than applied unconditionally to all vocabulary matrices.

Eqs. (7) and (10) give parameter counts on a single vocabulary matrix; bilateral input+lm_head configs sum over the matrices actually adapted.

4.4 Four Adapter Topologies

Vocabulary-boundary adapter placement and tied/untied structure jointly determine effective matrices. We compare four topologies in Table 2, each usable with either A-LoRA or Vocab LoRA.

Tied models naturally correspond to T3; untied models can implement T1/T2/T4, with T3 as an artificial tying ablation. Downstream experiments separately compare affine-only and hidden-LoRA-conditioned scenarios to distinguish standalone small-budget capacity from remaining incremental gain when hidden LoRA is already present.

4.5 Parameter Budget Matching

For fair comparison between A-LoRA and Vocab LoRA, given Vocab rank r_W , we take parameter-matched aff rank

$$r_{\text{aff}}(r_W) = \left\lfloor r_W \cdot \frac{V+d}{2d} \right\rfloor \approx \left\lfloor r_W \cdot \frac{V}{2d} \right\rfloor. \quad (11)$$

For example, at $V/d \approx 31$, Vocab rank-1 corresponds to aff rank ≈ 15 . Internal-layer LoRA base-lines allocate rank under the same total trainable parameter count; details appear in Sections A.7 and A.10.

Subsequent experiments therefore test whether A-LoRA is more efficient than Vocab LoRA, whether the E/U split translates into actual placement differences, and whether gains come from placement and parameterization rather than a larger training budget.

5 Downstream LoRA Post-Training Experiments

§3 shows that A-LoRA better matches output / tied boundary residuals in full Base→Instruct post-training, and §4 turns this into parameterization and topology. This section tests four questions in downstream SFT: whether A-LoRA is more cost-effective than vocabulary-dimensional LoRA; whether vocabulary-boundary layers are valuable standalone adaptation sites; which side is stronger under standalone small-budget adaptation, input or lm_head; and which side provides more effective remaining incremental gain when hidden LoRA is already present, and whether tied models can preserve mergeable structure.

5.1 Validation Paths

Experiments are organized around the four paths in Table 3, rather than enumerating all adapter combinations. The first two paths test parameter efficiency and the adaptation value of vocabulary-boundary layers themselves; the last two distinguish standalone from hidden-LoRA-conditioned regimes and check shared mergeable form on tied models.

The main text focuses on major variants: hidden LoRA, vocab-only LoRA, A-LoRA-only input/lm_head/bilateral, hidden + input/lm_head A-LoRA, and mergeable tied A-LoRA. Except for full fine-tune sanity probe and frozen base, all experiments freeze base weights and train only the corresponding adapters; full definitions appear in Table 16.

5.2 Data, Metrics, and Reproducibility

Main experiments use a 25k subset from MiniMind instruction data (Gong, 2025); external validation uses UltraChat (Ding et al., 2023) and two 100k instruction datasets from MiniMind. Downstream

Table 3: How downstream experiments correspond to prior analysis and method design.

Question	Main comparison	Results
Is hidden-space affine more efficient than W-form?	A-LoRA-only vs vocab-only LoRA, parameter budget matched	§6.1
Are vocabulary-boundary layers more than small internal LoRA substitutes?	A-LoRA-only vs parameter-comparable single-layer LoRA	§6.2
Under standalone small budget, is input or <code>lm_head</code> stronger?	A-LoRA-only input vs <code>lm_head</code> , across tied/untied and scales	§6.3
With hidden LoRA present, which side adds remaining gain; can tied models stay mergeable?	hidden LoRA + input/ <code>lm_head</code> ; mergeable tied A-LoRA	§6.4; Appendix A.9

results uniformly report final validation loss (lower is better) and trainable parameter count; main tables retain only aggregate results answering the questions in Table 3.

Downstream fine-tuning is implemented with Hugging Face Transformers (Wolf et al., 2020) and PEFT (Mangrulkar et al., 2022). Hidden LoRA defaults to q/k/v/o projections and MLP up/down/gate projections in every layer, rank 8, $\alpha = 16$; A-LoRA uses vocabulary-boundary hidden-space affine rank, commonly rank 8/16 and $s = \alpha/r \in \{8, 64\}$. Key placement comparisons are reported on 0.5B/0.6B with seeds 42/43/44; larger-model results serve as scale extrapolation checks; remaining reproducibility details appear in Sections A.10 and A.12.

6 Results and Analysis

This section reports downstream LoRA comparison results according to the validation questions of §5. All metrics are final validation loss (lower is better); main results use the 25k-scale instruction-tuning set, with external validation on two 100k-scale instruction datasets.

6.1 A-LoRA Is More Parameter-Efficient Than Vocabulary-Dimensional LoRA

Table 4 compares A-LoRA with vocab-only LoRA applied directly to vocabulary matrices. Both train

Table 4: Parameter-efficiency comparison between A-LoRA and vocab-only LoRA (main instruction-tuning set).

Model	Config	A-LoRA	Vocab LoRA	Param ratio
Qwen2.5-0.5B	rank 2	1.879	1.883	~1/137
Qwen2.5-0.5B	rank 4	1.852	1.816	~1/152
Qwen3-0.6B	rank 2	1.494	1.529	~1/120
Qwen3-0.6B	rank 4	1.469	1.495	~1/133
Qwen2.5-1.5B	aff r16 vs vocab r4	1.473	1.538	~1/24
Qwen3-1.7B	aff r16 vs vocab r4	0.981	1.145	~1/18

Table 5: A-LoRA-only vs parameter-comparable single-layer LoRA (Qwen3-0.6B, main instruction-tuning set).

Config	Params	eval loss
affine_input r16, $s=64$	34k	1.419
affine_input r16, $s=32$	34k	1.426
affine_input_lm_head r16, $s=64$	67k	1.320
single-layer LoRA qkvo, mid layer	41k	1.473
single-layer LoRA q, layer 0	49k	1.626
single-layer LoRA q, mid layer	49k	1.667
single-layer LoRA q, last layer	49k	1.780

only vocabulary-boundary layers, without hidden LoRA. Because vocab-only LoRA must learn V -dimensional factors, its parameter count is one to two orders of magnitude higher than A-LoRA; yet at low budget, affine_input already reaches comparable or lower validation loss. On Qwen3-0.6B in particular, A-LoRA rank 2/4 is lower than vocab-only LoRA by 0.035/0.026 with roughly 120–130 $\times$ fewer parameters; on 1.5B/1.7B, A-LoRA rank 16 uses only about 1/18–1/24 of the parameters yet beats vocab-only rank 4 by 0.065/0.164.

6.2 Vocabulary-Boundary Layers Are Not Small Substitutes for Single-Layer LoRA

The second question is whether vocabulary-boundary layers are valuable adaptation sites in their own right, rather than merely “a few extra parameters.” Table 5 uses same-scale small-budget comparisons on Qwen3-0.6B: unilateral affine_input has only 34k parameters, fewer than the strongest single-layer LoRA at 41k, yet validation loss is lower by 0.054; bilateral affine_input_lm_head uses slightly more parameters and lowers loss by a further 0.153. This shows vocabulary-boundary layers provide an adaptation channel distinct from any single internal layer.

6.3 Standalone Small-Budget Placement: Input Side Is Stronger

Table 6 reports affine-only settings: with no Transformer internal adapters trained, affine_input is

Table 6: Affine-only placement comparison: input side is stronger without hidden LoRA.

Model	tied?	input	lm_head	Δ
Qwen2.5-0.5B	Y	1.816	1.907	-0.091
Qwen3-0.6B	Y	1.460	1.621	-0.161
Qwen2.5-1.5B	Y	1.502	1.608	-0.106
Qwen2.5-7B	N	1.258	1.392	-0.134

Table 7: Placement comparison on top of hidden LoRA: lm_head side is the more effective supplement.

Model	hidden	input + hidden	lm_head + hidden	Δ
Qwen2.5-0.5B	1.302	1.297	1.288	-0.009
Qwen3-0.6B	1.101	1.097	1.093	-0.004
Qwen2.5-1.5B	—	1.050	1.044	-0.006
Qwen3-1.7B	—	0.7362	0.7328	-0.0034
Qwen2.5-7B	—	0.918	0.913	-0.005

significantly stronger than affine_lm_head across scales and tied/untied structures. This is not limited to tied models; Qwen2.5-7B is untied, and the input side is still lower by 0.134. Here we test standalone small-budget adaptation capacity, not reproduction of boundary residuals left after full Base→Instruct post-training.

6.4 With Hidden LoRA Present: lm_head Provides More Effective Remaining Gain

Once full hidden LoRA is added, the trend reverses. Table 7 shows that on top of hidden LoRA, affine_lm_head beats affine_input on 0.5B/0.6B small models, 1.5B/1.7B tied models, and the 7B untied model. With hidden LoRA present, marginal gain from input-side adapters is usually only 0.003–0.005; the lm_head side stably adds 0.005–0.014.

This reversal is stable across repeats. Table 8 gives means over seeds 42/43/44 on 0.5B/0.6B; all six seed–model configs satisfy affine_lm_head + hidden beating affine_input + hidden, with cross-seed variance smaller than the gap between sides. Larger-model comparisons at 1.5B/1.7B/7B use a single seed but follow the same direction as the multi-seed small models.

The key is that the two regimes target different objectives. §3 characterizes the residual structure left at vocabulary-boundary layers after full Base→Instruct post-training: LM head / tied matrix change resembles a hidden-space affine map, while untied input embedding does not. A-LoRA-only training is an extremely low-budget standalone adaptation scenario without hidden LoRA: input-

Table 8: Multi-seed placement results with hidden LoRA (seeds 42/43/44, main instruction-tuning set).

Model	input + hidden	lm_head + hidden	Δ
Qwen2.5-0.5B	1.297±0.000	1.288±0.001	-0.009
Qwen3-0.6B	1.097±0.001	1.093±0.001	-0.003

side perturbation propagates through all Transformer layers and therefore has stronger standalone fitting capacity, and is not equivalent to fitting full post-training boundary residuals. After hidden LoRA is added, internal adapters already absorb much hidden-representation adaptation, and remaining vocabulary-boundary increment more closely resembles output-side / tied residuals from full post-training; then the affine-friendly structure of §3 begins to dominate actual gain. In other words, A-LoRA placement is not determined by static structure alone, but jointly by target residual type and existing adaptation capacity.

Control experiments support this explanation. With hidden LoRA only at layer 0, lm_head already beats input by 0.05–0.09; removing layer 0 while keeping other layers yields results nearly identical to full hidden LoRA. Further ablations removing the last or middle layers preserve lm_head advantage. This shows input/hidden redundancy is not a layer-0 local phenomenon, but global functional overlap dispersed across internal layers.

Tied compatibility and external validation. For tied-embedding models, we also implement mergeable tied A-LoRA: input and lm_head share the same affine adapter, with transposed action on the output side so the trained adapter can merge back into a single tied embedding. This shows vocabulary-boundary adaptation need not rely on decoupled input/output matrices and can remain compatible with tied-weight deployment. In effect, mergeable tied + hidden on Qwen3-0.6B is close to affine_lm_head + hidden and beats hidden LoRA alone; merge equivalence checks and redundancy analysis appear in Section A.9.

On two external 100k instruction datasets, UltraChat 100k and MiniMind random 100k, A-LoRA gains are smaller than on main data but overall flat or slightly positive; in low-rank queues combined configs usually match hidden LoRA or are slightly lower. External validation therefore supports a conservative conclusion: A-LoRA does not cause degradation across data distributions and provides stable small gains under low rank or main-

data settings.

Overall, downstream experiments yield a finer conclusion than simple “which side is stronger”: A-LoRA’s hidden-space parameterization is more cost-effective than vocabulary-dimensional LoRA; vocabulary-boundary layers have standalone adaptation capacity; placement must distinguish standalone small-budget adaptation from remaining incremental gain when hidden LoRA is present; for tied models, shared mergeable form preserves tied structure without noticeably sacrificing performance.

7 Conclusion

This paper re-examines vocabulary-boundary layers long skipped by default in LoRA placement. Input embeddings and output LM heads can technically be adapted, but under standard Vocab LoRA they face joint constraints from V -dimensional parameter cost, weight tying, vocabulary parallelism, and merge requirements. The key question is therefore not “can they be updated,” but whether they can participate in post-training in a parameter-efficient and structurally sound way. We propose A-LoRA, parameterizing vocabulary-matrix updates as a shared low-rank affine transform in hidden space so adapter parameters depend mainly on d rather than V .

Around this parameterization, we provide structural conditions for applicability. Post-training vector-space analysis on 30 Base→Instruct model pairs shows vocabulary entry and exit are not symmetric; budget-matched offline fitting further shows A-LoRA’s advantage comes from matching this change structure, not from simply adding a few parameters. This finding limits A-LoRA from a mere parameter-compression trick to a conditional placement method.

Downstream LoRA post-training further reveals the conditionality of placement. A-LoRA beats vocab-only LoRA and parameter-comparable single-layer LoRA at small budgets, showing vocabulary-boundary layers provide an adaptation channel distinct from any single internal layer; yet there is no unconditional superiority between input and output sides. When used standalone, `affine_input` is significantly stronger than `affine_lm_head` on both tied and untied models, because input perturbation propagates through the entire Transformer and tests standalone small-budget fitting capacity; once combined with full

hidden LoRA, internal adapters already absorb much hidden-representation change, and remaining boundary increment more closely resembles output-side / tied residuals from full post-training, so `affine_lm_head` advantage echoes the prior structural analysis. Mergeable tied A-LoRA further shows this design can preserve tied structure while maintaining performance. Overall, vocabulary-boundary layers should not be excluded from LoRA design space by default, nor added symmetrically and unconditionally. Our conclusion is that vocabulary-boundary LoRA becomes an effective supplement to internal-layer LoRA at controllable cost only when adapter form, placement topology, target residual type, and existing adaptation capacity align.

Limitations

Our downstream experiments focus mainly on instruction tuning. Although they cover Qwen models of different scales and tied/untied structures and include validation on two external instruction datasets, we have not systematically evaluated code generation, long context, multilingual transfer, knowledge injection, or domain continued pretraining. A-LoRA’s gain magnitude, optimal topology, and hyperparameter choices in those tasks therefore remain to be verified.

Our static Base→Instruct vector-space analysis covers multiple model families, but downstream training evidence concentrates mainly on the Qwen series. Future work should perform equally scaled downstream validation on more families such as Llama, Gemma, and DeepSeek to confirm cross-family stability of placement reversal and mergeable tied conclusions.

Our downstream results mainly report validation loss and trainable parameter count, and do not systematically report generation quality, task accuracy, human preference, or safety-related metrics. Whether small gains in validation loss consistently translate into user-perceptible generation quality improvements therefore requires task-specific evaluation.

We focus on vocabulary-boundary placement, parameter efficiency, and mergeable tied form, and do not treat inference latency, serving throughput, or cross-device tensor-parallel communication as a full engineering benchmark. Related implementation constraints are noted in the method and appendix, but system evaluation for production de-

ployment remains future work.

Ethical Considerations

This work performs method analysis and empirical evaluation on publicly available pretrained models and common downstream task datasets. It does not involve human subjects or personally sensitive data. To the authors’ knowledge, there are no additional ethical risks requiring disclosure.

References

AI@Meta. 2024. *The Llama 3 herd of models*. Preprint, arXiv:2407.21783.

Harsh Bihany, Shubham Patel, and Ashutosh Modi. 2025. *LoRMA: Low-rank multiplicative adaptation for LLMs*. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10113–10133, Vienna, Austria. Association for Computational Linguistics.

DeepSeek-AI. 2024. *DeepSeek-V3 technical report*. Preprint, arXiv:2412.19437.

DeepSeek-AI. 2025. *DeepSeek-V3.1 model release*. <https://huggingface.co/deepseek-ai/DeepSeek-V3.1>. Official model card; Accessed 2026-05-17.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. *QLoRA: Efficient finetuning of quantized LLMs*. In *Advances in Neural Information Processing Systems*, volume 36.

Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. *Enhancing chat language models by scaling high-quality instructional conversations*. arXiv preprint arXiv:2305.14233.

Gemma Team. 2024. *Gemma 2: Improving open language models at a practical size*. Preprint, arXiv:2408.00118.

Gemma Team. 2025. *Gemma 3 technical report*. Preprint, arXiv:2503.19786.

Jingyao Gong. 2025. *MiniMind: Train small language models from scratch*. <https://github.com/jingyaogong/minimind>. Official repository and dataset release.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. *LoRA: Low-rank adaptation of large language models*. In *International Conference on Learning Representations*.

Hugging Face. 2026a. *LoRA: PEFT developer guide*. https://huggingface.co/docs/peft/en/developer_guides/lora. Accessed 2026-05-17.

Hugging Face. 2026b. *Trainable tokens: PEFT documentation*. https://huggingface.co/docs/peft/package_reference/trainable_tokens. Accessed 2026-05-17.

Hakan Inan, Kevin Khassanchi, and Richard Socher. 2017. *Tying word vectors and word classifiers: A loss framework for language modeling*. In *International Conference on Learning Representations*.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. 2022. *PEFT: State-of-the-art parameter-efficient fine-tuning methods*. <https://github.com/huggingface/peft>. Software library.

Meta. 2024. *Llama 3.2 model card*. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md. Official model card; Accessed 2026-05-17.

Ofir Press and Lior Wolf. 2017. *Using the output embedding to improve language models*. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 157–163, Valencia, Spain. Association for Computational Linguistics.

Qwen Team. 2025a. *Qwen2.5 technical report*. Preprint, arXiv:2412.15115.

Qwen Team. 2025b. *Qwen3 technical report*. Preprint, arXiv:2505.09388.

Qwen Team. 2026. *Qwen3.5: Towards native multi-modal agents*. <https://qwen.ai/blog?id=qwen3.5>. Official model-family release and citation; Accessed 2026-05-17.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. *Megatron-LM: Training multi-billion parameter language models using model parallelism*. arXiv preprint arXiv:1909.08053.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. *Transformers: State-of-the-art natural language processing*. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. *AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning*. In *International Conference on Learning Representations*.

A Supplementary Material

This appendix supplements experimental setup and per-model values for §3, §4, and §5, for separate consultation of post-training vector-space static analysis, A-LoRA budget probes, and downstream LoRA post-training experiments.

A.1 Notation and Main-Text Correspondence

The main text uses V for vocabulary size and d for hidden dimension. Base/Instruct weights are denoted X and Y ; $\Delta W = Y - X$ characterizes post-training change of vocabulary row vectors in hidden space. A-LoRA low-rank factors P, Q and Vocab LoRA factors B, C are independent; β is A-LoRA’s hidden-dimensional bias.

Table 9: Correspondence between main-text objects and appendix experiment tables.

Main text	Question	Appendix section / table
§3 Table 1	Aggregate R_K^2 , $A_{L,K}$	Table 11; full data in Section A.13
§3.4	aff/W equal budget	Table 12; full data in Section A.13
§4.4 Table 2	T1–T4 definitions	Section A.6
§5.1	Downstream fine-tuning	Section A.10

A.2 BI Model Pairs and Exclusion Criteria

Inclusion criteria: auditable public Base→Instruct weight pairs; full embed_tokens and lm_head (or tied shared) weights readable; V, d consistent after vocabulary alignment.

Main analysis group: non-excluded **30 pairs** = main 26 + extended 4 (DeepSeek-V3/V3.1, Qwen3-30B-A3B, Qwen3.5-35B-A3B), of which 17 tied and 13 untied. Full registry has 35 pairs; 5 excluded pairs appear in Section A.3. Table 19 lists Base/Instruct names, tier, and tied status for all 30 pairs. Model families include Qwen2.5/Qwen3/Qwen3.5 (Qwen Team, 2025a,b, 2026), Llama-3.x (AI@Meta, 2024; Meta, 2024), Gemma-2/3 (Gemma Team, 2024, 2025), and DeepSeek-V3/V3.1 (DeepSeek-AI, 2024, 2025).

A.3 Excluded Models

The following 5 pairs are **not included** in the 30-pair main narrative:

- **Gemma-4** (4 pairs): row-norm gauge in weight files (Base E near-unit-sphere) + Per-Layer Embeddings (PLE) architecture, so raw

Table 10: 30-pair BI main analysis group summarized by model family (n = pairs included from that family).

Family	n	tied	untied	Notes
Qwen3 / Qwen3.5	11	7	4	includes 2 MoE pairs
Qwen2.5	7	3	4	
Llama-3.x	4	2	2	
Gemma-2 / Gemma-3	6	6	0	
DeepSeek-V3 series	2	0	2	extended
Total	30	17	13	

E/U differ in convention from standard models; full-vocab affine R_E^2 far below main-stream (0.67–0.78 vs. 0.99+).

- **Gemma-3-1B** (1 pair): abnormal full-vocab $E R^2$ (≈ 0.38), BI rewrite magnitude too large.

A.4 Effective Affine Subspace Analysis (§3 Details)

Construction: $Z = [X, \mathbf{1}_V]$; SVD on Z ; take first K left singular vectors Q_K with cumulative energy $\eta = 95\%$. On raw $\Delta W = Y - X$, compute $R_{\text{aff},K}^2 = \|Q_K Q_K^\top \Delta W\|_F^2 / \|\Delta W\|_F^2$.

Auxiliary metrics: sample-dimension SVD on ΔW yields σ_i, u_i ; define $a_i(K)$, r_L , $A_{L,K}$, $R_{\text{aff},K}^{2,(L)} = r_L A_{L,K}$. Under centered convention, $D = Y_c - X_c$, $P = X_c(A - I)$, $P/D = \|P\|_F^2 / \|D\|_F^2$ (see Table 21).

Table 11: Untied 13 BI pairs: median effective subspace metrics ($\eta = 95\%$). $R_K^2 \equiv R_{\text{aff},K}^2$.

Side	r_5	$A_{5,K}$	$R_K^{2,(5)}$	$R_K^{2,(10)}$	R_K^2	P/D
E	1.9%	0.048	0.07%	0.10%	3.6%	4.9%
U	22.2%	0.829	13.3%	16.0%	33.0%	31.9%

Key comparison: in all 13/13 untied pairs, $R_K^2(\text{U}) > R_K^2(\text{E})$ and P/D also holds 13/13; tied 17 pairs have $R_K^2 \approx 32\%$, $A_{5,K} \approx 0.82$, on par with untied U. Per-model 43 rows appear in Table 20.

A.5 A-LoRA vs Vocab LoRA Budget Probe (§3 Details)

Parameter budget probe protocol: offline fitting simulation on observed post-training change, not adapter training. Given $D = Y_c - X_c$, optimal full affine yields $P = X_c(A - I)$, $R = D - P$. At W rank k , Vocab LoRA fits with k -rank W-form; A-LoRA fits with $r_{\text{aff}}(k)$ -rank hidden factors per Eq. (11). Report $\text{aff}/W = \text{gain}_{\text{aff}}/\text{gain}_W$.

Table 12: A-LoRA vs Vocab LoRA grouped summary (median over 30 pairs).

Group	n	aff/W@1	win@1	aff/W@4	win@4	aff@W1
Untied U	13	2.18×	12/13	1.79×	10/13	15
Untied E	13	0.34×	1/13	0.33×	0/13	15
Tied E=U	17	3.00×	15/17	1.81×	14/17	37

Hybrid ($P + R$) adds limited incremental value to the U-side main narrative and is not primary evidence in the main text. 30-pair model list and per-model budget appear in Tables 19 and 21.

A.6 Topology Configuration Details

The four topologies of Table 2 combine orthogonally with adapter type in downstream LoRA post-training. Experimental roles of each combination appear in Table 13.

Table 13: Topology $\times$ adapter recommendation level. A-only = without hidden LoRA; +HL = combined with internal hidden LoRA.

Topology	A-only		+HL	
	tied	untied	tied	untied
T1 Input-only	***	***	secondary	secondary
T2 Output-only	control	control	***	***
T3 Shared / mergeable	**	—	***	—
T4 Decoupled	**	**	control	control

A.7 Parameter Budget Matching and Baselines

Within vocabulary-boundary comparison: fix Vocab rank r_W ; A-LoRA takes $r_{\text{aff}} = \lfloor r_W(V + d)/(2d) \rfloor$, ensuring $P_{\text{aff}} \approx P_W$ (Eqs. (7)–(11)).

Comparison with internal-layer LoRA: let internal LoRA act on standard modules (Q/K/V/O + MLP), with parameter count $P_{\text{int}} = L_{\text{mod}} \cdot r_{\text{int}}(V_{\text{mod}} + d)$. Boundary configs satisfy $P_{\text{border}} + P_{\text{int}} = P_{\text{total}}$ or fixed P_{total} sweeps; specific ranks appear in Section A.10.

A.8 Method Details: Forward Pass, LoRMA+ Relation, and Merge

Vocab LoRA forward. For embedding lookup,

$$e_x = M_0[x, :] + \frac{\alpha}{r} B[x, :] C; \quad (12)$$

for LM head,

$$z = M_0 h + \frac{\alpha}{r} B C h. \quad (13)$$

In Eq. (12), only row x of B is activated, yielding row-sparse gradients along vocabulary dimension; Eq. (13) produces dense gradients over all V rows.

A-LoRA forward. For token x ,

$$e_x = M_0[x, :](I_d + \frac{\alpha}{r} P Q) + \beta \quad (14)$$

$$= M_0[x, :] + M_0[x, :] \frac{\alpha}{r} P Q + \beta. \quad (15)$$

For hidden $h \in \mathbb{R}^d$,

$$z = M_0(I_d + \frac{\alpha}{r} P Q)h + \mathbf{1}_V(\beta^\top h) \quad (16)$$

$$= M_0 h + M_0 \frac{\alpha}{r} P Q h + (\beta^\top h) \mathbf{1}_V. \quad (17)$$

Here $\beta^\top h$ broadcasts to full-vocabulary logits on the output side, parameterizing bias-type change in §3.3.

Relation to LoRMA+. LoRMA+ writes effective weights as $W_{\text{eff}} = (I + \frac{\alpha}{r} B A) W_0$ (Bihany et al., 2025); A-LoRA’s multiplicative part $M_0(I_d + \frac{\alpha}{r} P Q)$ is formally similar. The difference is that A-LoRA targets $V \times d$ vocabulary matrices, exploits dimensional asymmetry $V \gg d$ by placing adapter parameters in hidden space, and additionally includes $\mathbf{1}_V \beta^\top$; its applicability is constrained by post-training vector-space analysis of §3, rather than serving as a general multiplicative LoRA substitute on internal square matrices.

Shared merge. For shared A-LoRA, the effective shared matrix can be merged offline as

$$W_{\text{merged}}[i, :] := W_0[i, :](I_d + \frac{\alpha}{r} P Q) + \beta, \quad (18)$$

where $i \in \{1, \dots, V\}$. After merge, inference introduces no extra latency; under decoupled topology, $E_{\text{eff}} \neq U_{\text{eff}}$ and merge to a single tied matrix is generally impossible.

A.9 Mergeable Tied A-LoRA Details

Mergeable tied A-LoRA addresses deployment constraints in tied-embedding models. It shares the same hidden-space affine adapter on input and lm_head sides, with transposed action on the output side so effective weights after training can still merge back into one tied embedding matrix. After merge per Eq. (18), we check element-wise difference between online adapter logits and merged-weight logits; maximum absolute error in main experiments is about 3×10^{-5} – 6×10^{-5} , within floating-point error.

This form can also avoid redundancy when bilateral independent adapters are combined with hidden LoRA on tied models: part of the input branch is already absorbed by internal hidden LoRA, while

the output branch provides remaining capacity before final logits. Table 14 illustrates that this engineering form does not cause obvious loss in parameters or performance; main-text conclusions still rely primarily on placement reversal and structural analysis of §3.

Table 14: Parameter and effect calibration for mergeable tied A-LoRA (Qwen3-0.6B, main instruction-tuning set).

Config	Params	eval loss
hidden LoRA	5.05M	1.101
affine_input + hidden	5.06M	1.097
affine_lm_head + hidden	5.06M	1.093
affine_input_lm_head + hidden	5.08M	1.095
mergeable tied + hidden	5.06M	1.094

A.10 Downstream LoRA Post-Training Setup

This section lists unified reproducibility settings for downstream LoRA post-training. The main text reports only aggregate results supporting primary claims; key raw results appear in Section A.11.

Table 15: Unified downstream LoRA post-training configuration.

Item	Setting
Main data	sft_t2t_mini_25k subset from MiniMind instruction data, 24k train + 1k eval.
External validation	UltraChat 100k (100k train + 2k eval) and MiniMind random 100k (100k train + 2k eval).
Base models	Qwen2.5-0.5B/1.5B/3B/7B and Qwen3-0.6B/1.7B/4B/8B; covers tied and untied embedding structures.
Training	Except full fine-tune sanity probe, base weights frozen and only adapters trained; default 1 epoch, max sequence length 1024, effective batch size is typically 16, learning rate 2×10^{-4} , cosine decay, warmup ratio 0.03.
hidden LoRA	Default on q/k/v/o and MLP up/down/gate projections in every layer, rank 8, $\alpha = 16$; low-rank sweep uses rank 1/2/4.
A-LoRA	Default vocabulary-boundary hidden-space affine rank 16, $\alpha = 128$ ($s = \alpha/r = 8$); A-LoRA-only and tied alignment experiments additionally sweep $s \in \{16, 32, 64\}$ or rank 32.
Vocab LoRA	W-form LoRA on embed_tokens/lm_head, commonly rank 1/2/4, as vocabulary-dimensional parameter-efficiency control.
Seeds and metrics	Default seed 42; key placement, tied, and mergeable experiments add seeds 43/44. Uniformly report final validation loss (eval loss, lower is better) and trainable parameter count.

Table 16: Downstream LoRA / A-LoRA variant definitions.

Variant	Train location	Purpose
hidden LoRA	Transformer block projections	Standard PEFT main baseline
single-layer LoRA	Single decoder layer projections	Same-scale small-budget control
hidden + Vocab LoRA	hidden LoRA + embed_tokens/lm_head W-form	Vocab-dim LoRA parameter-efficiency control
A-LoRA-only input	embed_tokens A-LoRA	Standalone small-budget primary config
A-LoRA-only lm_head	lm_head A-LoRA	Affine-only placement attribution control
A-LoRA-only input+lm_head	embed_tokens + lm_head A-LoRA	Pure vocabulary-boundary bilateral control
hidden + input A-LoRA	hidden LoRA + input A-LoRA	Input-side redundancy check
hidden + lm_head A-LoRA	hidden LoRA + output A-LoRA	Primary config combined with hidden LoRA
hidden + input+lm_head A-LoRA	hidden LoRA + bilateral A-LoRA	Bilateral redundancy control
mergeable tied A-LoRA	tied shared A-LoRA, transposed on output	Mergeable tied config avoiding bilateral redundancy

A.11 Downstream Experiment Raw Result Tables

Tables 17–18 give key raw results corresponding to the four main experimental lines in the main text, including model, config, trainable parameter count, and final validation loss. To keep the appendix concise, we retain only representative run-level results supporting main claims. Parameter counts follow actual configs: A-LoRA input counts unilateral embed_tokens adapter; Vocab LoRA counts bilateral embed_tokens+lm_head W-form adapters; all counts include trainable parameters and bias, rounded to thousands or millions.

These raw results use the same training settings and evaluation protocol as main-text tables: all values are final validation loss after 1 epoch, lower is better.

Table 17: Raw results for parameter efficiency and vocabulary-boundary value.

Evidence line	Model	Config	Params	eval loss
Parameter efficiency	Qwen3-0.6B	A-LoRA input r2	5.1k	1.494
Parameter efficiency	Qwen3-0.6B	Vocab LoRA r2	0.61M	1.529
Parameter efficiency	Qwen3-0.6B	A-LoRA input r4	9.2k	1.469
Parameter efficiency	Qwen3-0.6B	Vocab LoRA r4	1.23M	1.495
Parameter efficiency	Qwen2.5-1.5B	A-LoRA input r16	50.7k	1.473
Parameter efficiency	Qwen2.5-1.5B	Vocab LoRA r4	1.23M	1.538
Parameter efficiency	Qwen3-1.7B	A-LoRA input r16	67.6k	0.981
Parameter efficiency	Qwen3-1.7B	Vocab LoRA r4	1.24M	1.145
Boundary value	Qwen3-0.6B	A-LoRA input r16, $s=64$	34k	1.419
Boundary value	Qwen3-0.6B	single-layer LoRA qkvo, mid	41k	1.473
Boundary value	Qwen3-0.6B	single-layer LoRA q, layer 0	49k	1.626
Boundary value	Qwen3-0.6B	single-layer LoRA q, mid	49k	1.667

Table 18: Raw results for standalone placement, hidden-LoRA-conditioned placement, and mergeable tied.

Evidence line	Model / condition	baseline / input	lm_head / mergeable	Conclusion
standalone	Qwen2.5-0.5B	1.816	1.907	input stronger
standalone	Qwen3-0.6B	1.460	1.621	input stronger
standalone	Qwen2.5-1.5B	1.502	1.608	input stronger
standalone	Qwen2.5-7B untied	1.258	1.392	input stronger
+hidden LoRA	Qwen2.5-0.5B	1.297	1.288	lm_head stronger
+hidden LoRA	Qwen3-0.6B	1.097	1.093	lm_head stronger
+hidden LoRA	Qwen2.5-1.5B	1.050	1.044	lm_head stronger
+hidden LoRA	Qwen3-1.7B	0.7362	0.7328	lm_head stronger
+hidden LoRA	Qwen2.5-7B untied	0.918	0.913	lm_head stronger
multi-seed	Qwen2.5-0.5B seed 43/44	1.297 / 1.297	1.289 / 1.288	same direction
multi-seed	Qwen3-0.6B seed 43/44	1.096 / 1.097	1.094 / 1.093	same direction
mergeable tied	Qwen3-0.6B + hidden	hidden 1.101	mergeable 1.094	effect preserved
mergeable tied	Qwen3-0.6B seeds 43/44	—	1.095 / 1.095	stable
external	UltraChat 100k 1.5B / 1.7B	hidden 1.075 / 1.058	mergeable 1.075 / 1.058	no degradation

A.12 Implementation Notes

A-LoRA implementation. Eq. (14) can compute $M_0[x, :] \frac{\alpha}{r} PQ + \beta$ after lookup, avoiding materializing full M_{eff} .

Vocabulary-parallel tensor parallelism. In vocabulary-parallel settings, vocabulary-related matrices can be partitioned along the V dimension (Shoeybi et al., 2019). Thus Vocab LoRA’s $B \in \mathbb{R}^{V \times r}$ shards with the vocabulary dimension, whereas A-LoRA’s P, Q, β contain no V dimension and can be replicated or sharded along hidden dimensions.

Merge and serving. T3 shared A-LoRA can merge per Eq. (18) for single-matrix serving; T4 decoupled requires loading E_{eff} and U_{eff} separately.

A.13 Per-Model Raw Data Tables

The following three tables provide per-model raw data for main-text and above aggregate tables; placed at appendix end to avoid cross-column whitespace in two-column layout.

Table 19: 30-pair BI model list (Base→Instruct).

Base	Instruct	tier	tied	d	V
DS-V3	DS-V3	ext.	N	7168	129280
DS-V3.1	DS-V3.1	ext.	N	7168	129280
G-2-27B	G-2-27B	main	Y	4608	256000
G-2-2B	G-2-2B	main	Y	2304	256000
G-2-9B	G-2-9B	main	Y	3584	256000
G-3-12B	G-3-12B	main	Y	3840	262208
G-3-27B	G-3-27B	main	Y	5376	262208
G-3-4B	G-3-4B	main	Y	2560	262208
L-3.1-70B	L-3.1-70B	main	N	8192	128256
L-3.1-8B	L-3.1-8B	main	N	4096	128256
L-3.2-1B	L-3.2-1B	main	Y	2048	128256
L-3.2-3B	L-3.2-3B	main	Y	3072	128256
Q2.5-0.5B	Q2.5-0.5B	main	Y	896	151936
Q2.5-1.5B	Q2.5-1.5B	main	Y	1536	151936
Q2.5-14B	Q2.5-14B	main	N	5120	152064
Q2.5-32B	Q2.5-32B	main	N	5120	152064
Q2.5-3B	Q2.5-3B	main	Y	2048	151936
Q2.5-72B	Q2.5-72B	main	N	8192	152064
Q2.5-7B	Q2.5-7B	main	N	3584	152064
Q3-0.6B	Q3-0.6B	main	Y	1024	151936
Q3-1.7B	Q3-1.7B	main	Y	2048	151936
Q3-14B	Q3-14B	main	N	5120	151936
Q3-30B-A3B	Q3-30B-A3B	ext.	N	2048	151936
Q3-4B	Q3-4B	main	Y	2560	151936
Q3-8B	Q3-8B	main	N	4096	151936
Q3.5-0.8B	Q3.5-0.8B	main	Y	1024	248320
Q3.5-2B	Q3.5-2B	main	Y	2048	248320
Q3.5-35B-A3B	Q3.5-35B-A3B	ext.	N	2048	248320
Q3.5-4B	Q3.5-4B	main	Y	2560	248320
Q3.5-9B	Q3.5-9B	main	N	4096	248320

Table 20: Per-model effective affine subspace metrics (43 rows, $\eta = 95\%$). Tied models: E=U row only; untied models: separate E/U rows. $R_K^2 \equiv R_{\text{aff},K}^2$.

Base model	Side	r_5	$A_{5,K}$	$A_5/(K/V)$	R_K^2	$R_K^{2,(5)}$	$R_K^{2,(10)}$
Q3-0.6B	E	21.6%	0.749	$137.1 \times$	34.8%	16.20%	19.68%
Q3-1.7B	E	18.4%	0.790	$67.7 \times$	32.2%	14.50%	16.85%
Q3-4B	E	16.6%	0.817	$58.3 \times$	32.2%	13.52%	16.27%
Q3-8B	E	2.3%	0.088	$4.9 \times$	6.1%	0.20%	0.26%
Q3-8B	U	14.5%	0.802	$34.6 \times$	32.0%	11.64%	14.29%
Q3-14B	E	2.8%	0.079	$4.9 \times$	5.7%	0.22%	0.29%
Q3-14B	U	13.4%	0.852	$29.6 \times$	32.4%	11.42%	14.06%
Q3-30B-A3B	E	0.9%	0.057	$5.6 \times$	3.4%	0.05%	0.09%
Q3-30B-A3B	U	17.8%	0.747	$66.6 \times$	33.0%	13.28%	16.05%
Q3.5-0.8B	E	9.7%	0.523	$189.4 \times$	11.0%	5.09%	5.85%
Q3.5-2B	E	8.2%	0.684	$118.3 \times$	11.6%	5.64%	6.12%
Q3.5-4B	E	7.4%	0.667	$91.8 \times$	11.5%	4.91%	5.36%
Q3.5-9B	E	1.6%	0.029	$2.2 \times$	2.5%	0.05%	0.07%
Q3.5-9B	U	9.7%	0.731	$55.3 \times$	20.2%	7.12%	8.30%
Q3.5-35B-A3B	E	2.1%	0.020	$3.8 \times$	1.5%	0.04%	0.06%
Q3.5-35B-A3B	U	10.5%	0.665	$105.1 \times$	20.0%	6.98%	8.15%
Q2.5-0.5B	E	29.1%	0.878	$271.7 \times$	31.4%	25.53%	26.22%
Q2.5-1.5B	E	68.5%	0.756	$92.6 \times$	57.1%	51.75%	52.72%
Q2.5-3B	E	63.0%	0.752	$67.5 \times$	53.8%	47.37%	48.55%
Q2.5-7B	E	4.8%	0.015	$1.6 \times$	1.5%	0.07%	0.10%
Q2.5-7B	U	48.8%	0.839	$48.5 \times$	47.0%	40.98%	41.90%
Q2.5-14B	E	3.0%	0.023	$1.6 \times$	2.4%	0.07%	0.10%
Q2.5-14B	U	49.4%	0.834	$29.5 \times$	51.5%	41.20%	42.87%
Q2.5-32B	E	3.2%	0.021	$1.6 \times$	2.5%	0.07%	0.11%
Q2.5-32B	U	52.2%	0.855	$30.3 \times$	53.0%	44.60%	45.67%
Q2.5-72B	E	1.5%	0.032	$1.3 \times$	3.6%	0.05%	0.08%

Continued on next page

(continued from Table 20)

Base model	Side	r_5	$A_{5,K}$	$A_5/(K/V)$	R_K^2	$R_K^{2,(5)}$	$R_K^{2,(10)}$
Q2.5-72B	U	51.2%	0.889	21.1×	51.8%	45.52%	46.14%
L-3.2-1B	E	26.0%	0.884	69.3×	45.8%	22.99%	25.82%
L-3.2-3B	E	22.0%	0.886	45.6×	44.4%	19.47%	22.02%
L-3.1-8B	E	1.7%	0.050	2.2×	3.7%	0.09%	0.12%
L-3.1-8B	U	22.9%	0.829	32.7×	38.2%	18.97%	21.90%
L-3.1-70B	E	1.2%	0.068	1.5×	6.1%	0.08%	0.12%
L-3.1-70B	U	22.2%	0.844	17.0×	38.6%	18.78%	20.95%
G-2-2B	E	47.5%	0.660	90.2×	39.6%	31.35%	32.67%
G-2-9B	E	32.8%	0.944	85.4×	38.8%	30.97%	32.91%
G-2-27B	E	35.7%	0.871	65.0×	40.6%	31.11%	33.58%
G-3-4B	E	9.4%	0.879	111.1×	13.5%	8.27%	9.52%
G-3-12B	E	7.5%	0.836	71.9×	12.9%	6.27%	7.27%
G-3-27B	E	5.1%	0.866	53.4×	10.3%	4.39%	4.81%
DS-V3	E	1.9%	0.048	1.1×	4.8%	0.09%	0.14%
DS-V3	U	22.8%	0.366	8.3×	13.2%	8.36%	8.96%
DS-V3.1	E	1.3%	0.050	1.1×	4.9%	0.06%	0.10%
DS-V3.1	U	16.1%	0.465	10.6×	12.8%	7.48%	8.16%

Table 21: Per-model A-LoRA budget probe (30 pairs). P/D = centered linear share; aff/W@1 = aff/W explained-gain ratio at W rank=1; aff@W1 = parameter-matched aff rank on U side.

Base model	tier	tied	V/d	P/D (E)	P/D (U)	aff/W@1 (E)	aff/W@1 (U)	aff@W1
DS-V3	ext.	N	18.0	0.060	0.122	0.25×	0.75×	9
DS-V3.1	ext.	N	18.0	0.061	0.112	0.29×	1.15×	9
G-2-27B	main	Y	55.6	0.320	0.320	3.00×	3.00×	28
G-2-2B	main	Y	111.1	0.369	0.369	1.30×	1.30×	56
G-2-9B	main	Y	71.4	0.269	0.269	2.60×	2.60×	36
G-3-12B	main	Y	68.3	0.123	0.123	2.18×	2.18×	34
G-3-27B	main	Y	48.8	0.099	0.099	2.58×	2.58×	24
G-3-4B	main	Y	102.4	0.101	0.101	4.32×	4.32×	51
L-3.1-70B	main	N	15.7	0.082	0.478	0.36×	2.18×	8
L-3.1-8B	main	N	31.3	0.048	0.411	0.26×	2.82×	16
L-3.2-1B	main	Y	62.6	0.408	0.408	3.04×	3.04×	31
L-3.2-3B	main	Y	41.8	0.402	0.402	2.87×	2.87×	21
Q2.5-0.5B	main	Y	169.6	0.257	0.257	1.58×	1.58×	85
Q2.5-1.5B	main	Y	98.9	0.568	0.568	0.93×	0.93×	49
Q2.5-14B	main	N	29.7	0.047	0.486	0.23×	1.18×	15
Q2.5-32B	main	N	29.7	0.049	0.515	0.28×	1.09×	15
Q2.5-3B	main	Y	74.2	0.554	0.554	0.98×	0.98×	37
Q2.5-72B	main	N	18.6	0.073	0.481	0.34×	1.08×	9
Q2.5-7B	main	N	42.4	0.032	0.462	0.18×	1.16×	21
Q3-0.6B	main	Y	148.4	0.311	0.311	6.00×	6.00×	74
Q3-1.7B	main	Y	74.2	0.296	0.296	4.18×	4.18×	37
Q3-14B	main	N	29.7	0.079	0.319	0.52×	4.30×	15
Q3-30B-A3B	ext.	N	74.2	0.041	0.311	1.48×	5.17×	37
Q3-4B	main	Y	59.4	0.298	0.298	4.99×	4.99×	30
Q3-8B	main	N	37.1	0.073	0.308	0.54×	4.45×	19
Q3.5-0.8B	main	Y	242.5	0.119	0.119	4.27×	4.27×	121
Q3.5-2B	main	Y	121.2	0.115	0.115	3.45×	3.45×	61
Q3.5-35B-A3B	ext.	N	121.2	0.020	0.207	0.59×	5.25×	61
Q3.5-4B	main	Y	97.0	0.120	0.120	3.04×	3.04×	49
Q3.5-9B	main	N	60.6	0.029	0.210	0.35×	5.04×	30